12. Isolation Forest

Outliers are easier to isolate than normal data.

Previous Section:

11. Multiple Imputation by Chained Equation (MICE)

Contents:

In the **Outliers in Data** section, we introduced statistical methods such as **Z-Score** and **Interquartile Range (IQR)** for detecting anomalous observations. These methods work well for simple datasets but become less effective when dealing with multiple features or complex data distributions.

In this section, we introduce **Isolation Forest**, a machine learning algorithm specifically designed for outlier detection.

Unlike statistical methods that measure how far a data point is from the majority, Isolation Forest identifies anomalies by repeatedly isolating observations through random partitions. Since outliers are typically rare and very different from normal observations, they tend to be separated much earlier than the rest of the data.

1. What is an Isolation Forest?

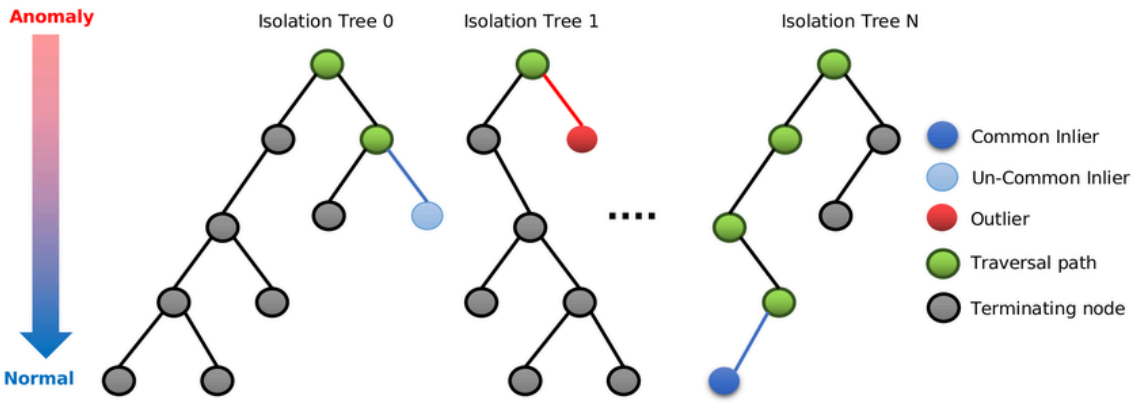
Isolation Forest is an **unsupervised machine learning algorithm** for detecting outliers, especially in high-dimensional datasets. Its fundamental idea is simple:

> ***Outliers are easier to isolate than normal data points.***

Instead of modeling what normal data looks like, Isolation Forest repeatedly partitions the data until each observation is isolated.

1.1. How It Works

Isolation Forest builds many **Isolation Trees**, each created using completely random splits.



For every tree:

- Randomly select one feature.
- Randomly choose a split value within that feature's range.
- Divide the data into two groups.
- Repeat the process until each data point becomes isolated.

Unlike traditional decision trees, Isolation Trees do **not** search for the best split. Every split is chosen randomly, making the algorithm both simple and computationally efficient.

1.2. Outlier Detection Logic

After constructing many Isolation Trees, the algorithm measures **how many splits (path length)** are required to isolate each data point.

- Data points isolated after only a few splits have a **short path length** and are likely to be outliers.
- Data points requiring many splits have a **long path length** and are considered normal observations.

The average path length across all trees is then converted into an anomaly score.

- **Short average path → Outlier**
- **Long average path → Normal data**

1.3. Why Is It Called an Isolation Forest?

The algorithm does **not** rely on a single Isolation Tree. Instead, it builds **many Isolation Trees**, each using different random feature selections and random split values. Together, these trees form an **Isolation Forest**.

Because each tree is generated randomly, one tree may accidentally isolate a normal point early. However, if the **same observation is consistently isolated after only a few splits across many trees**, it is much more likely to be a genuine outlier.

Using many trees makes the algorithm far more robust and reliable than relying on a single randomly generated tree.

2. Isolation Forest with Python

Scikit-Learn provides an implementation of the Isolation Forest algorithm through the **IsolationForest** class. In this example, we will generate a synthetic dataset containing **1,000 samples**, intentionally introduce **20% outliers**, train an Isolation Forest model, and visualize several of the generated Isolation Trees.

- **Step 1. Import Libraries**

```
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs
from sklearn.ensemble import IsolationForest
from sklearn.tree import plot_tree
```

- Step 2. Generate a Dataset

```
np.random.seed(42)

# Normal samples
X_normal, _ = make_blobs(
    n_samples=800,
    centers=1,
    cluster_std=2,
    random_state=42
)

# Artificial outliers
X_outlier = np.random.uniform(
    low=-10,
    high=10,
    size=(200, 2)
)

X = np.vstack((X_normal, X_outlier))

df = pd.DataFrame(X, columns=["Feature1", "Feature2"])

print(df.head(20))
```

- Step 3. Visualize the Dataset

```
plt.figure(figsize=(6,6))
plt.scatter(df["Feature1"], df["Feature2"], s=15)
```

```
plt.title("Dataset with 20% Outliers")
plt.show()
```

- Step 4. Train the Isolation Forest

```
model = IsolationForest(
    n_estimators=100,
    contamination=0.20,
    random_state=42
)

model.fit(df)
```

- Step 5. Detect Outliers

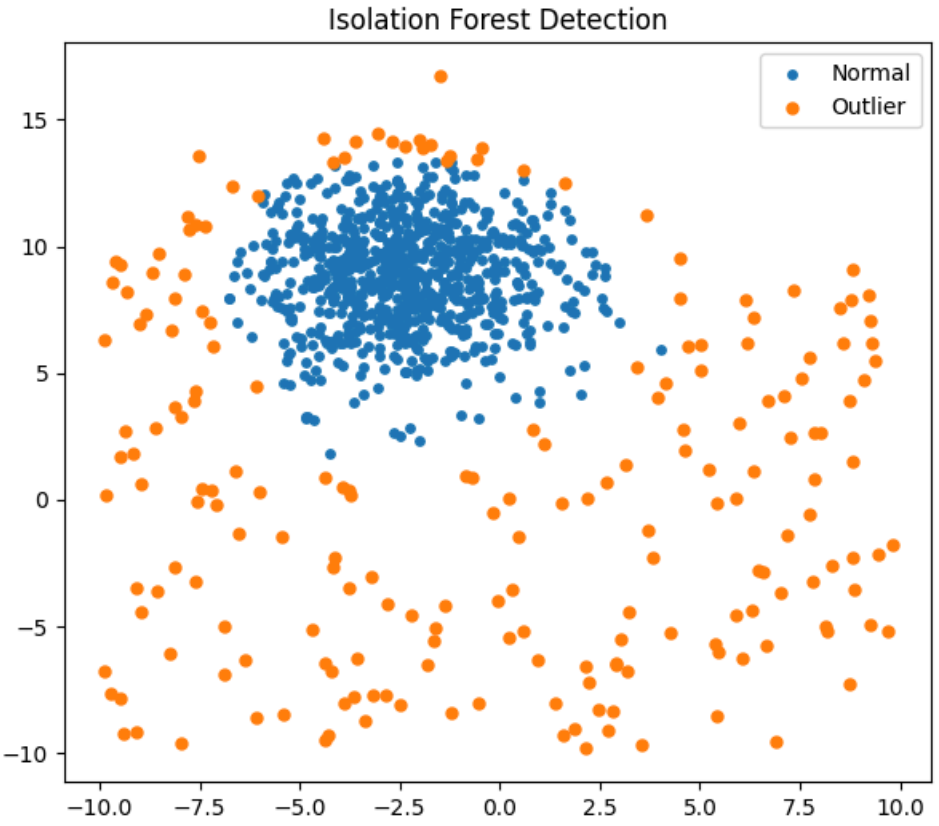
```
df["Prediction"] = model.predict(df)

# -1 = Outlier
#  1 = Normal
print(df[df['Prediction'] == -1].head(20)
```

```
# Count the detected samples.
print(df["Prediction"].value_counts())
```

- Step 6. Visualize the Results

```
plt.figure(figsize=(7,6))

normal = df[df["Prediction"] == 1]
outlier = df[df["Prediction"] == -1]

plt.scatter(
```

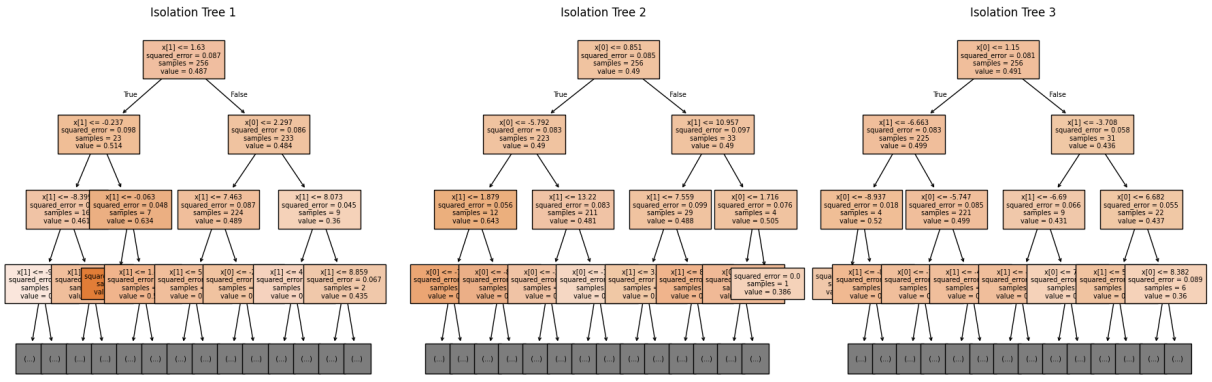
```
    normal["Feature1"],
    normal["Feature2"],
    s=15,
    label="Normal"
)

plt.scatter(
    outlier["Feature1"],
    outlier["Feature2"],
    s=25,
    label="Outlier"
)

plt.legend()
plt.title("Isolation Forest Detection")
plt.show()
```



- Step 7. Visualize Several Isolation Trees

Each estimator inside the forest is an individual **Isolation Tree**.

```
fig, axes = plt.subplots(1, 3, figsize=(18,6))

for i, ax in enumerate(axes):
    plot_tree(
        model.estimators_[i],
        max_depth=3,
        filled=True,
        fontsize=7,
        ax=ax
    )

    ax.set_title(f"Isolation Tree {i+1}")

plt.tight_layout()
plt.show()
```



For better interpretation, consider drawing only one tree at the time

Notice that each tree has a different structure because every split is selected randomly. Although individual trees vary, they work together to identify observations that are consistently isolated after only a few splits.

Summary

In this section, we introduced **Isolation Forest**, an unsupervised machine learning algorithm designed specifically for outlier detection. Unlike statistical techniques such as Z-Score and IQR, Isolation Forest identifies anomalies by

repeatedly partitioning the data with random feature selections and random split values. Since outliers are easier to isolate than normal observations, they typically have much shorter path lengths within the trees.

We learned how an **Isolation Tree** isolates individual observations and why combining many randomly generated trees into an **Isolation Forest** produces more reliable results.

Finally, we implemented the algorithm in Python using Scikit-Learn, generated a synthetic dataset containing outliers, detected anomalous observations, and visualized several Isolation Trees to better understand how the algorithm works internally.

References

https://www.geeksforgeeks.org/machine-learning/anomaly-detection-using-isolation-forest/

https://www.geeksforgeeks.org/machine-learning/what-is-isolation-forest/

https://machinelearningmastery.com/anomaly-detection-with-isolation-forest-and-kernel-density-estimation/

Next Section:

ROC-AUC Curve